

mmdb - ex 09 (Indexing)

1.1 Indexing = • Increase execution speed of retrieval queries by using index the db can instantly jump to the rows it needs instead of checking every row.

drawbacks: • more storage

• slow down insert, delete, update

1.3. B-tree: - a balanced tree with sorted keys in internal nodes and leaves
- Finds data quickly by narrowing the search range step-by-step

Hash: - hash function $\rightarrow$ bucket $\rightarrow$ row location

- Jump directly to the bucket for a Key

K-d tree: - Recursive binary tree; each level split by one coordinate.

- Helps find nearby points or points in a region

Point quadtree: • Each node splits into 4 parts in 2D

• Stores point by recursively placing them into sub regions

1.2. Methods for reduction of dimensionality

• Efficiency in query processing

2.1 Characteristics: R-tree speeds up spatial search by grouping objects into MBR and skipping whole branches that cannot match the query.

Structure: - Leaf nodes store actual spatial objects or points

- Internal nodes store MBRs that cover all objects below them

- The tree is height balanced

2.2. Insert = • Start at the root

• For each internal node, choose the child whose MBR needs the smallest enlargement to fit new object

• Go down until a leaf is reached and insert the object

• If the node overflows, split it into two nodes

• Then update the MBRs on the path back to root

Delete = • Find leaf that contains the object and remove the object

• If the node becomes too empty, remove underfull nodes $\rightarrow$ reinsert entries elsewhere

• Update the MBRs on the way up

3.	Query type	without Index	with Index
	Exact queries	check every object and test	Use an index like B-tree to jump

	$X = q$	directly to the key
Range queries	Check every object and keep those with $d(q, x) \leq r$	Use similarity index and prune whole branches that cannot match
Nearest neighbor	Check every object, keep the smallest distance	Traverse the index and prune anything that cannot beat the current best